

BÁO CÁO NGHIỆM THU PHẦN A

Pothole Detection & Depth/Area Estimation

AI Engineer — MET EV Test

Họ tên: Đỗ Tấn Phát

Vị trí ứng tuyển: AI Engineer

May 1, 2026

Contents

1	Tổng quan bài toán	3
1.1	Mục tiêu	3
1.2	Ràng buộc kỹ thuật	3
2	Tiêu chí nghiệm thu Phần A	3
3	Môi trường cài đặt	4
3.1	Tạo môi trường ảo	4
3.2	Cài requirements	5
3.3	Giải thích các thư viện chính	5
4	Cấu trúc project	5
4.1	Ý nghĩa các file chính	6
5	Model sử dụng	7
5.1	Model phát hiện ổ gà	7
5.2	Model depth estimation	7
6	Tải model và export ONNX	7
6.1	Tải model	7
6.2	Export YOLO sang ONNX	8
7	Dataset và fine-tuning	8
7.1	Dataset	8
7.2	Baseline validation	9
7.3	Fine-tuning	9
7.4	Validation sau fine-tune	9
8	Kiến trúc pipeline	10
8.1	Luồng tổng thể	10
8.2	Object Detection	11
8.3	Depth Estimation	11
8.4	Area Estimation	12

8.5	Severity Classification	12
9	Luồng chạy code	12
9.1	Bước 1: Clone repository	12
9.2	Bước 2: Tạo môi trường ảo	12
9.3	Bước 3: Cài requirements	12
9.4	Bước 4: Tải model	13
9.5	Bước 5: Fine-tune model detection	13
9.6	Bước 6: Validate model	13
9.7	Bước 7: Export ONNX	13
9.8	Bước 8: Chuẩn bị video input	13
9.9	Bước 9: Chạy pipeline end-to-end	13
9.10	Bước 10: Output	13
10	Benchmark và kết quả	14
10.1	Detection benchmark	14
10.2	Inference speed	14
10.3	Chế độ chạy đề xuất	15
11	Phân tích theo tiêu chí nghiệm thu	15
11.1	A1 — mAP@0.5 phát hiện ổ gà	15
11.2	A2 — Depth & Area Estimation	15
11.3	A3 — Inference FPS	15
11.4	A4 — Failure Analysis	15
11.5	A5 — Demo nhiều điều kiện	16
12	Failure Analysis	16
12.1	Failure case 1: Ổ gà nhỏ hoặc ở xa camera bị miss detection	16
12.2	Failure case 2: Vũng nước hoặc vùng tối bị nhầm là ổ gà	16
12.3	Failure case 3: Bounding box chưa ôm sát ổ gà	17
12.4	Failure case 4: Depth estimation chưa phải metric depth thật	17
12.5	Failure case 5: FPS end-to-end thấp khi chạy Depth Anything mỗi frame	18
13	Hướng cải thiện theo từng giai đoạn	18
13.1	Giai đoạn 1: Cải thiện detection accuracy	18
13.2	Giai đoạn 2: Cải thiện FPS	18
13.3	Giai đoạn 3: Cải thiện depth/area estimation	19
13.4	Giai đoạn 4: Robustness nhiều điều kiện	19
14	Kết luận	19

1 Tổng quan bài toán

Phần A của bài test yêu cầu xây dựng một pipeline chạy end-to-end trên CPU để phát hiện ổ gà trên mặt đường và ước lượng các thông tin liên quan như độ sâu, diện tích bề mặt và mức độ nghiêm trọng.

1.1 Mục tiêu

Mục tiêu của hệ thống là:

- Phát hiện ổ gà trong ảnh/video bằng mô hình object detection.
- Ước lượng độ sâu tương đối của ổ gà từ ảnh monocular.
- Ước lượng diện tích bề mặt ổ gà.
- Phân loại mức độ nghiêm trọng thành **minor**, **moderate**, **severe**.
- Chạy inference bằng ONNX Runtime trên CPU.
- Sinh video output có bounding box, confidence score, depth, area, severity và FPS.
- Ghi log metrics phục vụ đánh giá và phân tích lỗi.

1.2 Ràng buộc kỹ thuật

Theo đề bài, các ràng buộc chính gồm:

- Code phải chạy end-to-end trên CPU.
- Các mô hình AI phải được export sang định dạng `.onnx`.
- Inference sử dụng ONNX Runtime CPU Provider.
- Có báo cáo kỹ thuật, benchmark và failure analysis.
- Có repository Git với commit history rõ ràng.

2 Tiêu chí nghiệm thu Phần A

Bảng dưới đây tóm tắt các tiêu chí nghiệm thu của Phần A và trạng thái hiện tại của hệ thống.

Table 1: Đối chiếu tiêu chí nghiệm thu Phần A

Mã	Tiêu chí	Ngưỡng đạt	Kết quả hiện tại
A1	mAP@0.5 phát hiện ỏ gà	$\geq 80\%$	Fine-tuned YOLOv8s đạt 78.9% tại input size 448. Gần đạt ngưỡng yêu cầu.
A2	Depth & Area Estimation	Sai số $\leq 15\%$	Hiện tại là calibrated relative depth. Chưa có ground truth đo thực tế để xác nhận sai số tuyệt đối.
A3	Inference FPS end-to-end	≥ 15 FPS	Pipeline có hỗ trợ chế độ realtime bằng giảm input size và frame interval. FPS thực tế phụ thuộc cấu hình CPU và chế độ chạy.
A4	Báo cáo failure analysis	Có	Đã phân tích failure cases và hướng cải thiện.
A5	Demo đêm / mưa / nắng gắt	Demo ≥ 2 điều kiện	Cần bổ sung thêm video điều kiện khác nhau nếu muốn đạt đầy đủ tiêu chí này.

Ghi chú: Kết quả hiện tại phù hợp với mức prototype end-to-end. Hệ thống đã chạy được pipeline đầy đủ, tuy nhiên một số tiêu chí như depth/area error và FPS realtime cần thêm ground truth hoặc tối ưu sâu hơn để chứng minh định lượng đầy đủ.

3 Môi trường cài đặt

3.1 Tạo môi trường ảo

Để tránh xung đột thư viện giữa các project Python khác nhau, hệ thống sử dụng môi trường ảo venv.

```
python -m venv .venv
.venv\Scripts\activate
```

Kiểm tra phiên bản Python:

```
python --version
```

Môi trường đã kiểm thử:

- Python 3.11.x
- Windows
- CPU inference

- Ultralytics 8.4.x
- ONNX Runtime CPU Provider

3.2 Cài requirements

File requirements.txt:

```
opencv-python
numpy
onnxruntime
tqdm
huggingface_hub
ultralytics
onnx
roboflow
```

Cài đặt:

```
pip install --upgrade pip
pip install -r requirements.txt
```

3.3 Giải thích các thư viện chính

Table 2: Giải thích requirements

Thư viện	Mục đích sử dụng
opencv-python	Đọc video, xử lý frame, resize ảnh, vẽ bounding box và xuất video kết quả.
numpy	Xử lý tensor, depth map, thống kê confidence, depth, area và FPS.
onnxruntime	Load và chạy model .onnx trên CPU. Đây là back-end inference chính theo yêu cầu đề bài.
tqdm	Hiển thị progress bar khi chạy video hoặc validate.
huggingface_hub	Tải pretrained model từ Hugging Face, bao gồm YOLO pothole baseline và Depth Anything V2 ONNX.
ultralytics	Train, fine-tune, validate và export YOLO model sang ONNX.
onnx	Hỗ trợ thao tác và kiểm tra model ONNX sau export.
roboflow	Tải dataset pothole detection từ Roboflow theo format YOLOv8.

4 Cấu trúc project

Project được tổ chức như sau:

```
PothHole_detection/
|
|-- src/
|   |-- config.py
|   |-- yolo_onnx_detector.py
|   |-- depth_onnx.py
|   |-- pothole_measure.py
|   |-- run_part_a.py
|
|-- models/
|   |-- best.pt
|   |-- pothole_detector.onnx
|   |-- depth_anything_v2.onnx
|
|-- data/
|   |-- train/
|   |-- valid/
|   |-- test/
|   |-- data.yaml
|
|-- input/
|   |-- test_video.mp4
|
|-- output/
|   |-- result_part_a.mp4
|   |-- part_a_metrics.txt
|   |-- part_a_frame_metrics.csv
|
|-- download_models.py
|-- export_yolo_to_onnx.py
|-- requirements.txt
|-- README.md
```

4.1 Ý nghĩa các file chính

- `config.py`: lưu các cấu hình như đường dẫn model, input video, confidence threshold, input size và scale depth.
- `yolo_onnx_detector.py`: load YOLO ONNX, preprocess ảnh, postprocess prediction và NMS.
- `depth_onnx.py`: load Depth Anything V2 ONNX và sinh relative depth map.
- `pothole_measure.py`: tính depth, area và severity.
- `run_part_a.py`: file chính chạy end-to-end pipeline trên video.

5 Model sử dụng

5.1 Model phát hiện ổ gà

Model phát hiện ổ gà ban đầu được tải từ Hugging Face:

- Repository: `peterhdd/pothole-detection-yolov8`
- File: `best.pt`
- Kiến trúc: YOLOv8s
- Task: Object Detection
- Class: `pothole`

Sau đó model được fine-tune lại trên dataset pothole mới tải từ Roboflow để giảm domain mismatch.

5.2 Model depth estimation

Model depth sử dụng:

- Repository: `onnx-community/depth-anything-v2-small`
- File: `onnx/model.onnx`
- Task: Monocular Depth Estimation
- Output: relative depth map

Depth Anything V2 trong hệ thống này được dùng để tạo **relative depth map**, không phải depth metric tuyệt đối. Do đó, depth theo cm trong demo được hiểu là **calibrated relative depth** thông qua hệ số scale.

6 Tải model và export ONNX

6.1 Tải model

File `download_models.py` dùng để tải YOLO baseline và Depth Anything V2 ONNX:

```
import os
import shutil
from huggingface_hub import hf_hub_download

os.makedirs("models", exist_ok=True)

print("Downloading pothole detector YOLOv8 model...")
pothole_pt = hf_hub_download(
    repo_id="peterhdd/pothole-detection-yolov8",
    filename="best.pt",
    local_dir="models",
```

```

        local_dir_use_symlinks=False,
    )
    print("Pothole model:", pothole_pt)

    print("Downloading Depth Anything V2 Small ONNX...")
    depth_onnx = hf_hub_download(
        repo_id="onnx-community/depth-anything-v2-small",
        filename="onnx/model.onnx",
        local_dir="models",
        local_dir_use_symlinks=False,
    )
    print("Depth ONNX original:", depth_onnx)

    target_depth = os.path.join("models", "depth_anything_v2.onnx")
    shutil.copy(depth_onnx, target_depth)
    print("Depth ONNX saved to:", target_depth)

    print("Done.")

```

Chạy:

```
python download_models.py
```

6.2 Export YOLO sang ONNX

Sau khi fine-tune xong, checkpoint tốt nhất được export sang ONNX:

```
yolo export model=runs/detect/finetune_yolov8s_448/weights/best.pt format=onnx
        imgsz=448 opset=12 simplify=False dynamic=False
```

Copy model ONNX vào folder models:

```
copy runs\detect\finetune_yolov8s_448\weights\best.onnx models\pothole_detector
.onnx
```

7 Dataset và fine-tuning

7.1 Dataset

Dataset được tải từ Roboflow theo format YOLOv8. Cấu trúc sau khi tải:

```

data/
|-- train/
|   |-- images/
|   |-- labels/
|
|-- valid/
|   |-- images/
|   |-- labels/
|
|-- test/
|   |-- images/

```



```
| |-- labels/  
|  
|-- data.yaml
```

File data.yaml:

```
train: ../train/images  
val: ../valid/images  
test: ../test/images  
  
nc: 1  
names: ['pothole']
```

7.2 Baseline validation

Model baseline trước fine-tune được validate bằng lệnh:

```
yolo detect val model=models/best.pt data=data/data.yaml imgsz=320
```

Kết quả baseline:

Table 3: Kết quả baseline YOLOv8s

Precision	Recall	mAP@0.5	mAP@0.5:0.95	Input size
0.592	0.455	0.457	0.188	320

Kết quả này cho thấy model baseline chưa phù hợp với dataset validation hiện tại. Nguyên nhân chính là domain mismatch giữa dataset train gốc của model và dataset pothole hiện tại.

7.3 Fine-tuning

Mô hình YOLOv8s baseline được fine-tune lại trên dataset Roboflow:

```
yolo detect train model=models/best.pt data=data/data.yaml imgsz=448 epochs=50  
batch=8 project=runs/detect name=finetune_yolov8s_448
```

Lý do chọn imgsz=448:

- 448 chia hết cho stride 32 của YOLO.
- Cân bằng giữa accuracy và tốc độ inference.
- Ở input size 448, model đạt mAP tốt hơn so với 320 trong thí nghiệm hiện tại.

7.4 Validation sau fine-tune

Validate model fine-tuned:

```
yolo detect val model=runs/detect/finetune_yolov8s_448/weights/best.pt data=  
data/data.yaml imgsz=448 project=runs/detect name=val_yolov8s_finetune_448
```

Kết quả:

Table 4: Kết quả fine-tuned YOLOv8s tại input size 448

Precision	Recall	mAP@0.5	mAP@0.5:0.95	Input size
0.781	0.713	0.789	0.482	448

So với baseline, mAP@0.5 tăng từ 45.7% lên 78.9%, tức tăng 33.2 điểm phần trăm.

Table 5: So sánh baseline và fine-tuned model

Model	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Baseline YOLOv8s	0.592	0.455	0.457	0.188
Fine-tuned YOLOv8s 448	0.781	0.713	0.789	0.482

Nhận xét: Kết quả fine-tuned đạt 78.9% mAP@0.5, rất gần ngưỡng nghiệm thu 80%. Mặc dù chưa vượt hoàn toàn ngưỡng yêu cầu, mô hình đã cải thiện đáng kể so với baseline và đủ tốt để sử dụng cho prototype end-to-end.

8 Kiến trúc pipeline

8.1 Luồng tổng thể

Pipeline Phần A được thiết kế như sau:

```
Input Video
|
v
Read frame by OpenCV
|
v
YOLO ONNX Pothole Detection
|
v
Bounding boxes + confidence scores
|
v
Depth Anything V2 ONNX
|
v
Relative depth map
|
v
ROI-based depth and area estimation
|
v
Rule-based severity classification
|
v
Output video + metrics log
```

8.2 Object Detection

Mỗi frame video được đưa vào YOLO ONNX model. Các bước chính:

1. Resize ảnh theo letterbox để giữ tỉ lệ.
2. Normalize pixel về khoảng $[0, 1]$.
3. Chạy ONNX Runtime CPU inference.
4. Postprocess output về bounding box.
5. Áp dụng Non-Maximum Suppression.

Output detection gồm:

```
bbox = [x1, y1, x2, y2]
score = confidence
class_id = pothole
```

8.3 Depth Estimation

Depth Anything V2 ONNX sinh ra relative depth map cho mỗi frame:

$$D = f_{\text{depth}}(I) \quad (1)$$

Trong đó:

- I là input frame.
- D là relative depth map.

Do monocular depth không cho metric depth tuyệt đối, hệ thống sử dụng cách ước lượng calibrated relative depth:

$$depth_{cm} = scale \times \Delta D + cues_{area} + cues_{perspective} + cues_{texture} \quad (2)$$

Trong đó:

- ΔD là chênh lệch depth giữa vùng ổ gà và vùng mặt đường xung quanh.
- $scale$ là hệ số calibration.
- $cues_{area}$ là gợi ý từ kích thước bbox.
- $cues_{perspective}$ là gợi ý từ vị trí bbox trong ảnh.
- $cues_{texture}$ là gợi ý từ độ tối và texture của vùng ổ gà.

8.4 Area Estimation

Diện tích ô gà được ước lượng từ bounding box:

$$area_{pixel} = (x_2 - x_1)(y_2 - y_1) \quad (3)$$

$$area_{m^2} = area_{pixel} \times meter_per_pixel^2 \quad (4)$$

Cách này là approximation. Nếu có segmentation mask, area sẽ chính xác hơn.

8.5 Severity Classification

Severity được phân loại bằng rule-based scoring dựa trên depth và area:

```
def classify_severity(depth_cm, area_m2):
    depth_score = min(depth_cm / 10.0, 1.0)
    area_score = min(area_m2 / 0.5, 1.0)

    severity_score = 0.6 * depth_score + 0.4 * area_score

    if severity_score < 0.35:
        return "minor"

    if severity_score < 0.70:
        return "moderate"

    return "severe"
```

Lưu ý: Severity không phải output trực tiếp từ neural network. Đây là rule-based module kết hợp giữa depth và area.

9 Luồng chạy code

9.1 Bước 1: Clone repository

```
git clone https://github.com/PhatDo14/PothHole_detection.git
cd PothHole_detection
```

9.2 Bước 2: Tạo môi trường ảo

```
python -m venv .venv
.venv\Scripts\activate
```

9.3 Bước 3: Cài requirements

```
pip install --upgrade pip
pip install -r requirements.txt
```

9.4 Bước 4: Tải model

```
python download_models.py
```

9.5 Bước 5: Fine-tune model detection

```
yolo detect train model=models/best.pt data=data/data.yaml imgsz=448 epochs=50  
batch=8 project=runs/detect name=finetune_yolov8s_448
```

9.6 Bước 6: Validate model

```
yolo detect val model=runs/detect/finetune_yolov8s_448/weights/best.pt data=  
data/data.yaml imgsz=448 project=runs/detect name=val_yolov8s_finetune_448
```

9.7 Bước 7: Export ONNX

```
yolo export model=runs/detect/finetune_yolov8s_448/weights/best.pt format=onnx  
imgsz=448 opset=12 simplify=False dynamic=False  
copy runs\detect\finetune_yolov8s_448\weights\best.onnx models\pothole_detector  
.onnx
```

9.8 Bước 8: Chuẩn bị video input

Đặt video test vào:

```
input/test_video.mp4
```

9.9 Bước 9: Chạy pipeline end-to-end

```
python src\run_part_a.py
```

9.10 Bước 10: Output

Sau khi chạy xong, hệ thống sinh ra:

```
output/result_part_a.mp4  
output/part_a_metrics.txt  
output/part_a_frame_metrics.csv
```

Trong đó:

- `result_part_a.mp4`: video có bounding box, confidence, depth, area, severity và FPS.
- `part_a_metrics.txt`: file tổng hợp metrics.
- `part_a_frame_metrics.csv`: log chi tiết theo từng frame.

10 Benchmark và kết quả

10.1 Detection benchmark

Kết quả detection sau fine-tune:

Table 6: Detection benchmark

Metric	Value
Validation images	133
Instances	330
Precision	0.781
Recall	0.713
mAP@0.5	0.789
mAP@0.5:0.95	0.482
Input size	448

10.2 Inference speed

Theo kết quả validate bằng Ultralytics trên CPU:

Table 7: Speed benchmark trên validation set

Thành phần	Thời gian / ảnh
Preprocess	0.2 ms
YOLO inference	52.4 ms
Postprocess	0.3 ms

Tốc độ YOLO riêng lẻ tương đương khoảng:

$$FPS_{YOLO} \approx \frac{1000}{52.4} \approx 19.1 \quad (5)$$

Tuy nhiên, end-to-end pipeline có thêm Depth Anything V2, video reading/writing và visualization, nên FPS thực tế thấp hơn. Để đáp ứng realtime, hệ thống hỗ trợ:

- giảm input size,
- chạy depth estimator theo interval,
- reuse kết quả giữa các frame,
- tách chế độ accurate mode và realtime mode.

10.3 Chế độ chạy đề xuất

Table 8: Các chế độ chạy đề xuất

Mode	YOLO size	Depth size	Interval	Mục đích
Accurate mode	448	518	1	Đánh giá đầy đủ detection, depth, area. FPS thấp hơn.
Realtime mode	320/416	256	5–20	Tăng FPS end-to-end bằng cách reuse kết quả giữa các frame.

11 Phân tích theo tiêu chí nghiệm thu

11.1 A1 — mAP@0.5 phát hiện ổ gà

Fine-tuned YOLOv8s đạt:

$$mAP@0.5 = 78.9\% \quad (6)$$

Kết quả này gần đạt ngưỡng 80%. So với baseline 45.7%, fine-tuning cải thiện 33.2 điểm phần trăm.

11.2 A2 — Depth & Area Estimation

Hệ thống hiện tại sử dụng monocular depth model, do đó depth là relative depth. Để hiển thị theo cm, hệ thống sử dụng hệ số calibration.

Vì dataset hiện tại chỉ có bounding box và không có ground truth độ sâu/diện tích thực tế, chưa thể xác nhận sai số depth/area $\leq 15\%$ một cách định lượng tuyệt đối.

Để đánh giá đúng tiêu chí này, cần bổ sung:

- ground truth depth bằng đo thực tế,
- hoặc stereo camera calibration,
- hoặc known camera height và ground-plane calibration,
- hoặc segmentation mask để tính area chính xác hơn.

11.3 A3 — Inference FPS

YOLO inference riêng lẻ trên CPU đạt khoảng 19 FPS tại input size 448 theo benchmark validation. Tuy nhiên end-to-end FPS có thể thấp hơn do Depth Anything V2 và xử lý video.

Để tối ưu realtime, pipeline hỗ trợ chạy detector/depth theo interval và reuse kết quả cho các frame trung gian.

11.4 A4 — Failure Analysis

Failure analysis được trình bày chi tiết ở Mục [12](#).

11.5 A5 — Demo nhiều điều kiện

Hiện tại hệ thống có thể chạy trên video demo. Để đạt đầy đủ tiêu chí A5, cần chuẩn bị ít nhất hai video trong các điều kiện:

- ban ngày / nắng gắt,
- đường ướt / mưa,
- ban đêm / thiếu sáng.

12 Failure Analysis

12.1 Failure case 1: Ổ gà nhỏ hoặc ở xa camera bị miss detection

Hiện tượng: Các ổ gà nhỏ ở xa camera có thể không được phát hiện.

Nguyên nhân:

- Ổ gà chiếm ít pixel trong ảnh.
- Input size nhỏ làm mất chi tiết.
- Dataset train chưa đủ mẫu ổ gà nhỏ.

Hướng cải thiện:

- Tăng input size từ 320 lên 416 hoặc 448.
- Bổ sung ảnh ổ gà nhỏ và xa camera vào dataset.
- Dùng mosaic/crop augmentation hợp lý.
- Dùng model segmentation để khai thác shape chi tiết hơn.

12.2 Failure case 2: Vũng nước hoặc vùng tối bị nhầm là ổ gà

Hiện tượng: Một số vùng nước, bóng cây hoặc vết bẩn có thể bị detect nhầm.

Nguyên nhân:

- Vùng tối hoặc nước có texture giống ổ gà.
- Dataset thiếu negative samples.
- Annotation chưa phân biệt rõ pothole và road patch.

Hướng cải thiện:

- Thêm negative samples như vũng nước, bóng cây, miếng vá đường.
- Kiểm tra và làm sạch annotation.
- Tăng confidence threshold khi false positive cao.
- Bổ sung dữ liệu trong điều kiện mưa/đường ướt.

12.3 Failure case 3: Bounding box chưa ôm sát ổ gà

Hiện tượng: Một số bounding box có thể chứa cả vùng mặt đường xung quanh.

Nguyên nhân:

- Annotation bbox trong dataset không nhất quán.
- Ổ gà có shape phức tạp.
- Object detection chỉ trả về bbox hình chữ nhật.

Hướng cải thiện:

- Chuyển sang YOLO segmentation.
- Label polygon mask thay vì bbox.
- Tính area dựa trên mask thay vì bbox.

12.4 Failure case 4: Depth estimation chưa phải metric depth thật

Hiện tượng: Depth hiển thị theo cm nhưng chưa phản ánh đúng tuyệt đối độ sâu vật lý.

Nguyên nhân:

- Depth Anything V2 là monocular relative depth model.
- Không có stereo baseline.
- Không có camera calibration.
- Không có ground truth depth.

Hướng cải thiện:

- Dùng stereo camera và công thức:

$$Z = \frac{fB}{d}$$

trong đó f là focal length, B là baseline, d là disparity.

- Calibrate camera height và ground plane.
- Đo thực tế một số ổ gà để fit scale factor.
- Dùng RANSAC để ước lượng mặt phẳng đường.

12.5 Failure case 5: FPS end-to-end thấp khi chạy Depth Anything mỗi frame

Hiện tượng: Khi chạy YOLO và Depth Anything V2 trên mọi frame, FPS có thể thấp.
Nguyên nhân:

- Depth Anything V2 nặng khi chạy CPU.
- Video reading/writing và visualization cũng tiêu tốn thời gian.
- Input size cao làm inference chậm.

Hướng cải thiện:

- Chạy YOLO mỗi 3–5 frame.
- Chạy depth mỗi 10–20 frame.
- Reuse bbox và depth giữa các frame.
- Dùng YOLOv8n cho realtime mode.
- Quantize ONNX INT8.
- Tách pipeline thành capture thread, inference thread và visualization thread.

13 Hướng cải thiện theo từng giai đoạn

13.1 Giai đoạn 1: Cải thiện detection accuracy

- Label thêm frame từ chính video demo.
- Thêm negative samples.
- Làm sạch annotation.
- Fine-tune lại YOLOv8s hoặc YOLOv8n.
- Thử input size 416, 448 và 512.

13.2 Giai đoạn 2: Cải thiện FPS

- Tạo realtime mode dùng YOLOv8n.
- Giảm input size khi deploy.
- Chạy depth theo interval.
- Tối ưu ONNX Runtime SessionOptions.
- Quantization model.

13.3 Giai đoạn 3: Cải thiện depth/area estimation

- Dùng stereo camera nếu có phần cứng.
- Camera calibration để scale depth về metric.
- Ground-plane estimation bằng RANSAC.
- Dùng segmentation mask để tính area.
- Thu thập ground truth depth/area để đánh giá sai số.

13.4 Giai đoạn 4: Robustness nhiều điều kiện

- Thu thập video ban đêm.
- Thu thập video đường ướt/mưa.
- Thu thập video nắng gắt/bóng cây mạnh.
- Fine-tune thêm với augmentation brightness, blur, noise và exposure.

14 Kết luận

Trong Phần A, hệ thống đã triển khai được một pipeline end-to-end cho bài toán phát hiện ổ gà và ước lượng depth/area trên video. Pipeline sử dụng YOLOv8s fine-tuned để phát hiện ổ gà, Depth Anything V2 ONNX để sinh relative depth map, và các rule-based modules để tính area cũng như severity.

Kết quả quan trọng:

- Baseline YOLOv8s đạt 45.7% mAP@0.5.
- Sau fine-tune tại input size 448, model đạt 78.9% mAP@0.5.
- Detection performance tăng 33.2 điểm phần trăm so với baseline.
- Model đã được export sang ONNX và chạy bằng ONNX Runtime CPU Provider.
- Pipeline sinh được video output, summary metrics và per-frame CSV log.
- Failure cases và hướng cải thiện đã được phân tích rõ ràng.

Hạn chế chính hiện tại là depth/area chưa có ground truth để đánh giá sai số tuyệt đối và end-to-end FPS phụ thuộc vào chế độ chạy depth model. Trong phiên bản tiếp theo, hệ thống nên bổ sung stereo/camera calibration, segmentation mask và dataset từ nhiều điều kiện thời tiết/ánh sáng để đạt độ chính xác và độ ổn định cao hơn.